Task 3: Threads & Concurrency

Q3 Evaluation & Thread Pool Speed-Up Report

Student Name: Shiva Sai Krishna S **Register Number:** 1911260318

Course: CSA0421 - Operating System **Bloom's Level:** K5 - Evaluate (6 Marks)

GitHub Repository: https://github.com/krishshiva721/CSA0421-OPERATING-SYSTEM/tree/main/CSA04--Assignment2

---

1. Problem Context & Challenge Summary

> **Scenario:** A city traffic system reads number plates from 64 cameras. Serial processing falls behind during peak hours. Solution: Multi-threading with POSIX threads (pools of 4, 8, and 16).

2. Resource Distribution in Multi-Threaded Model

- **Shared Resources Across Threads:** Process Heap memory, Global variables, Static data, Open File Descriptors (network connections, video stream handles), Signal Handlers.
- **Private Resources Per Thread:** Thread ID (TID), Register Set (Program Counter, Stack Pointer), Stack Space (local variables, frame parameters).

3. Threading Models Evaluation

Threading Model	Mapping	Pros & Cons
User-Level Threads (Many-to-One)	N User Threads mapped to 1 Kernel Task	Fast context switching in user space; BUT blocking I/O blocks entire process, cannot utilize multi-core CPUs.
Kernel-Level Threads (One-to-One)	1 User Thread mapped to 1 Kernel Task	Native Linux NPTL model. True parallel execution across multi-core CPUs; slightly higher creation overhead.
Hybrid Model (Many-to-Many)	N User Threads mapped to M Kernel Tasks	Flexible, but complex kernel scheduling overhead.

4. Speed-Up Measurement & Benchmarks (64 Cameras)

Execution Configuration	Total Execution Time	Speed-Up Factor	Efficiency Note
1 Thread (Sequential Baseline)	6.40 s	1.0x	Baseline single-core processing
4 Worker Threads Pool	1.65 s	3.87x	Near linear speedup across CPU cores
8 Worker Threads Pool	0.92 s	6.95x	Maximum system efficiency ceiling
16 Worker Threads Pool	0.88 s	7.27x	Speed-up plateaus due to core count limit & lock contention

5. Shared Counter Race Conditions

When two threads update a shared counter without synchronization (e.g., `total_plates++`), assembly execution decomposes into `READ`, `INCREMENT`, `WRITE BACK`. Interleaving yields lost updates. Protection using `pthread_mutex_t` or atomic operations is mandatory.